

To implement Zschokke's 1996 research on *Araneus diadematus* within **GTPyhop**, your GTPyhop code should reflect the shift from **opportunistic rule-based methods** to **predictable procedural decomposition** as follows.

The GTPyhop Implementation Pipeline

1. Define the State (state.py)

Zschokke's research tracks the spider's position and the thread network. You must maintain these as a graph.

```
import gtpyhop

class WebState(gtpyhop.State):
    def __init__(
        self, name
    ):
        super().__init__(name)
        self.spider_pos = "start_stick"
        self.nodes = {"start_stick", "end_stick"}
        self.threads = [] # List of (n1, n2, type)
        self.proto_hub_exists = False
```

2. Primitive Operators (operators.py)

These are the atomic movements documented by Zschokke (e.g., walking, anchoring, re-reeling dragline).

```
def walk(
    state, target
):
    state.spider_pos = target
    return state

def anchor(
    state, node
):
    state.nodes.add(node)
    return state

def lay_thread(
    state, n1, n2, t_type
):
    state.threads.append((n1, n2, t_type))
    return state
```

3. Methods for Variable Early Stages (Opportunistic Rules)

Zschokke found no single "perfect" way to build the proto-hub. Use GTPyhop's method

branching to represent this "random order" of behavioral patterns.

```
def establish_proto_hub
    # If hub exists, finish; else, choose one of the observed Zschokke patterns
    if state.proto_hub_exists:
        return []

    # Example: Choose between two observed behavioral rules
    if state.can_apply_rule_A():
        return [('rule_A_pattern'), ('establish_proto_hub',)]
    else:
        return [('rule_B_pattern'), ('establish_proto_hub',)]
```

4. Methods for Stereotyped Later Stages (Procedural)

Once the proto-hub is formed, switch to the rigid, repetitive decomposition documented in Zschokke's later work.

```
def build_orb
    # Stereotyped sequence
    return [('build_frame'), ('build_radii'), ('build_spirals',)]
```

Integrating Zschokke's 1996 Data Structures

To make this code authentic to the 1996 study:

- **Distance-as-Cost:** Zschokke used the distance walked as a proxy for metabolic energy. Implement a cost tracker in your operator definitions:

```
def walk
    distance = calculate_dist(start, end)
    state.energy_expended += distance
    state.spider_pos = end
    return state
```

- **Backtracking for Variability:** GTPyhop's built-in backtracking is perfect for modeling the "trial and error" behavior Zschokke observed. If a method (e.g., rule_A_pattern) fails to progress the state toward a proto_hub, the planner will automatically backtrack and attempt rule_B_pattern.